Deleting Matches from a Linked List

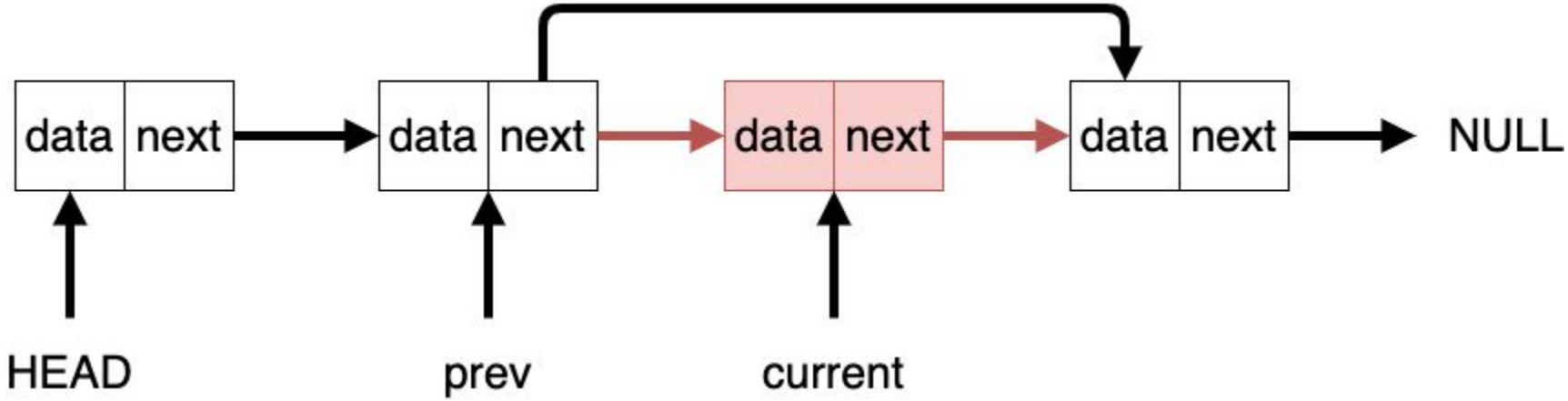
Kevin Browne

Deleting a matching node

- To delete a matching node from the list entirely, i.e. removing the node itself, is a bit more complicated!
  - And let's only consider deleting a single matching node to start with
- If the list is empty, we can do nothing
- If the head is a match, we can delete/free the head, and have the new list head become whatever head is pointing to

Deleting a matching node

- Where things get complicated is when we need to delete a node in the middle (or tail)
  - We would need the node ***previous*** to the node being deleted to point to the node that the node being deleted is pointing to (as in the diagram below)
  - Whew, try saying that 10 times fast!



Deleting a matching node

- We'll need to traverse the list while keeping track of both the previous node and the current node
  - Very similar to what we did when we deleted a node at the tail!
- If we reach a current matching node, we'll set the previous node to point to what the current node is pointing to instead
  - And we'll need to free the current node
- We could use a pass-by-reference bool value to keep track of whether a match was found at all

Deleting a matching node (an empty list or head match)

```
Node* delete_first_match(Node *head, int delete_value, bool *was_deleted)
{
  if (head == NULL) {
    *was_deleted = false;
    return NULL;
  }

  if (head->value == delete_value)
  {
    Node *temp = head->next;
    free(head);
    *was_deleted = true;
    return temp;
  }
```

Deleting a matching node continued... middle match case

```
  Node *current = head->next;
  Node *prev = head;

  while (current != NULL)
  {
    if (current->value == delete_value)
    {
      prev->next = current->next;
      free(current);
      *was_deleted = true;
      return head;
    }
    prev = current;
    current = current->next;
  }

  *was_deleted = false;
  return head;
}
```

Testing the function...

```
print_list(list1_head);
printf("\n");

bool deleted;
list1_head = delete_first_match(list1_head, 10, &deleted);
if (deleted) printf("A node with value 10 was deleted\n");
else printf("A node with value 10 was not deleted.\n");
printf("\n");
print_list(list1_head);
printf("\n");

list1_head = delete_first_match(list1_head, 5, &deleted);
if (deleted) printf("A node with value 5 was deleted\n");
else printf("A node with value 5 was not deleted.\n");
printf("\n");
print_list(list1_head);
printf("\n");
```

Test results…

```
Node 0: 5
Node 1: 7
Node 2: 14
Node 3: 7
Node 4: 7
Node 5: 14

A node with value 10 was not deleted.

Node 0: 5
Node 1: 7
Node 2: 14
Node 3: 7
Node 4: 7
Node 5: 14
```

Test results continued

```
A node with value 10 was not deleted

Node 0: 5
Node 1: 7
Node 2: 14
Node 3: 7
Node 4: 7
Node 5: 14

A node with value 5 was deleted

Node 0: 7
Node 1: 14
Node 2: 7
Node 3: 7
Node 4: 14
```

Let's try deleting matching middle and tail nodes...

```
list1_head = delete_first_match(list1_head, 14, &deleted);
if (deleted) printf("A node with value 14 was deleted\n");
else printf("A node with value 14 was not deleted.\n");
printf("\n");
print_list(list1_head);
printf("\n");

list1_head = delete_first_match(list1_head, 14, &deleted);
if (deleted) printf("A node with value 14 was deleted\n");
else printf("A node with value 14 was not deleted.\n");
printf("\n");
print_list(list1_head);
printf("\n");
```

Deleting matching nodes from the middle and tail of the list

```
Node 0: 7
Node 1: 14
Node 2: 7
Node 3: 7
Node 4: 14

A node with value 14 was deleted

Node 0: 7
Node 1: 7
Node 2: 7
Node 3: 14

A node with value 14 was deleted

Node 0: 7
Node 1: 7
Node 2: 7
```

What about deleting all matches?

- We could use this function to construct a crude (potentially inefficient) function to delete all matches
- We could call our delete match function until no matches are deleted
  - We could also count the number of deleted matches too!
- What makes this a potentially inefficient implementation though?
  - It will need to traverse the list ***again and again*** for each match!
  - What if there are many matches, all at the end of the list?

Delete all matches function

```
Node *delete_all_matches(Node *head, int delete_value, int *num_deleted)
{
  Node *current = head;
  bool deleted = false;
  *num_deleted = 0;

  do
  {
    current = delete_first_match(current, delete_value, &deleted);
    if (deleted) *num_deleted = *num_deleted + 1;
  } while (deleted);

  return current;
}
```

Deleting all matches from a list...

```
list_try before delete all matches...
Node 0: 8
Node 1: 5
Node 2: 8
Node 3: 6
Node 4: 8
Node 5: 8
Node 6: 7
Node 7: 8
Node 8: 8


list_try after delete all matches...
Node 0: 5
Node 1: 6
Node 2: 7

Elements deleted: 6
```